Program 2 - Scanner/parser

For this assignment, I used CHATGPT to help me understand how a recursive descent parser works and how to build one using Racket. I started by asking questions about the assignment because I wasn't sure where to begin. Throughout the process, I used AI to help me understand different functions, develop my scanner and parser, troubleshoot errors, and test my program. The following screenshot shows some of interaction with CHATGPT, including explanation, code suggestions, and improvements I made along the way.

Screenshot below :

Share

Works!, how do I actually start reading the source code and separating into token. how to do that in racket

Yess!! 😊 Now we're getting to the actual scanner!

In Racket, we can use **regular expressions (regex)** to recognize different parts of the source code, like numbers, variable names, and symbols.

But before writing the entire scanner, let's create one small helper function.

Step 2: Create `string-index`

This function will help us find the position of a character inside a string. We'll use it later when handling comments and strings.

Right below your `show-tokens` function, add:

racket

```
(define (string-index text character)
  (for/first ([i (in-range (string-length text))]
              #:when (char=? (string-ref text i) character))
    i))
```

What does this function do?

Let's break it down:

- `string-length` counts how many c cters are in the text.
- `in-range` gives us the positions of those characters, starting at 0.

Share

LEt start the aurally scanner function.

Yess!! 🎉 Now we're going to build the **actual scanner function** in Racket!

We'll do it step by step because this is one of the most important parts of your assignment.

Step 3: Start the `scan` function

Our scanner needs to recognize things like:

Plain text

```
program Demo {
    decl x as int;
    5 => x;
}
```

It needs to separate the code into numbers, identifiers, and symbols.

First, let's recognize those basic tokens

Under your `string-index` function, add:

racket

```
(define (scan source)
  (define pattern
```

Ask ChatGPT Instant

Plain text

```
}
```

Share

It needs to separate the code into numbers, identifiers, and symbols.

First, let's recognize those basic tokens

Under your `string-index` function, add:

racket

```
(define (scan source)
  (define pattern
    #px"^[0-9]+\\.[0-9]+|^[0-9]+|^[a-zA-Z][a-zA-Z0-9]*|^(=>|>=|<=|==|/=)|^[+*/;
{}()\\[\\],:<>~=-]")

  (let loop ([remaining source]
             [line 1]
             [tokens '()])
    (cond
      [(string=? remaining "")
       (reverse tokens)]

      [(char-whitespace? (string-ref remaining 0))
       (loop (substring remaining 1)
             (if (char=? (string-ref remaining 0) #\newline)
                 (add1 line)
                 line)
```

What is happening here?

Let's focus on the main ideas.

Ask ChatGPT Instant

Share

What is happening here?

Let's focus on the main ideas.

1. `define pattern`

This is our regular expression. It tells Racket what characters we're looking for.

For example:

- `[0-9]+` recognizes integers such as `5` or `100`.
- `[0-9]+\\.[0-9]+` recognizes decimals such as `5.5`.
- `[a-zA-Z][a-zA-Z0-9]*` recognizes identifiers such as `score` or `calculateBonus`.
- `=>`, `>=`, and `<=` recognize operators.

2. `let loop`

This repeatedly processes the remaining source code until there's nothing left.

It keeps track of the remaining text, the current line number, and the tokens we've collected.

3. `regexp-match`

This checks whether the beginning of the remaining text matches our pattern.

4. `cons` **and** `reverse`

We use `cons` to add each token to the front of our list. At the end, `reverse` puts the tokens back into the correct order.

Let's test it

Ask ChatGPT Instant

tokens back into the correct order.

Share

Let's test it

Under your scanner function, temporarily add:

```racket
(show-tokens (scan "5 => x;"))
```

Click **Run**.

You should see:

```
Type: integer | Value: 5 | Line: 1
Type: symbol | Value: => | Line: 1
Type: identifier | Value: x | Line: 1
Type: symbol | Value: ; | Line: 1
```

Notice that we haven't handled **comments (`--`) or backtick strings** yet. We'll add those next, since your professor requires both.

Try this first and tell me what DrRacket displays. Then we'll improve the scanner together!

Testing: Testfile-4.txt
parse: Syntax error on line 7: expected ;

Testing: Testfile-5.txt
scan: Syntax error on line 25

Testing: EmptyRoutine
(program EmptyRoutine () ((routine doNothing () int () () ())) ((empty)))

Testing: KeywordRoutine
...

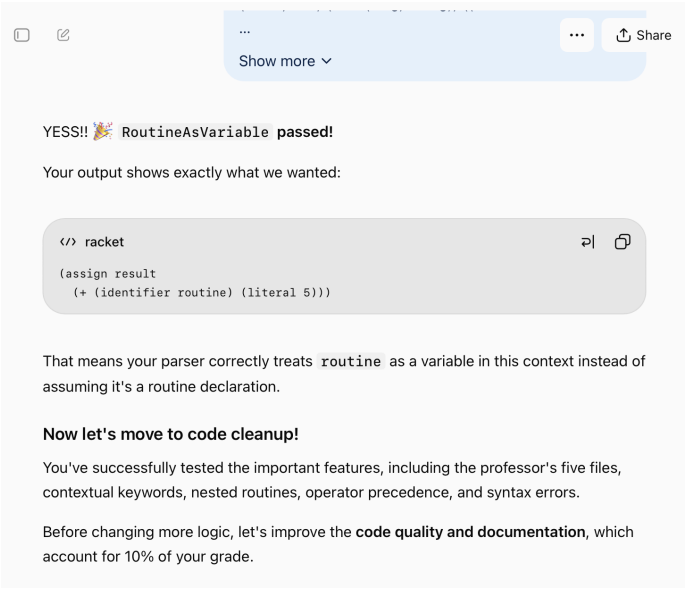
Show more

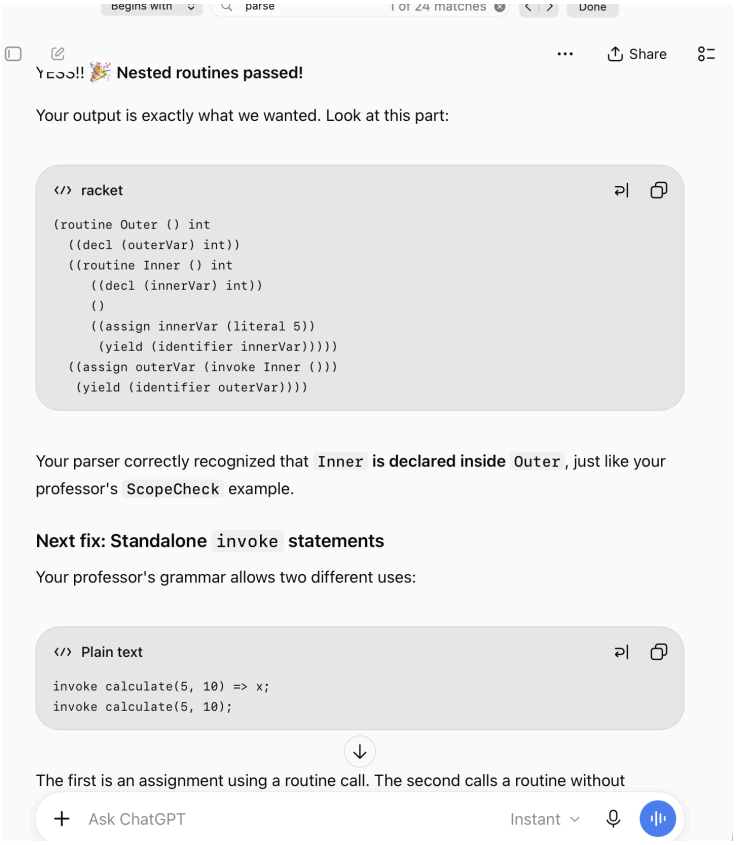
YESS!! 🎉 **Your final test run passed!**

All your previous results are still correct after adding the documentation comments.

Most importantly, the three syntax errors are **expected**, not problems with your parser:

- `Testfile-4.txt` — correctly detects a missing semicolon on line 7.
- `Testfile-5.txt` — correctly detects invalid syntax on line 25.
- `InvalidInvoke` — correctly detects an incomplete routine call on line 5.


...
Show more
Share
YESS!! 🎉 RoutineAsVariable passed!
Your output shows exactly what we wanted:
racket
(assign result
(+ (identifier routine) (literal 5)))
That means your parser correctly treats routine as a variable in this context instead of assuming it's a routine declaration.
Now let's move to code cleanup!
You've successfully tested the important features, including the professor's five files, contextual keywords, nested routines, operator precedence, and syntax errors.
Before changing more logic, let's improve the code quality and documentation, which account for 10% of your grade.


Begins with
parse
1 of 24 matches
Done
Share
YESS!! 🎉 Nested routines passed!
Your output is exactly what we wanted. Look at this part:
racket
(routine Outer () int
((decl (outerVar) int))
((routine Inner () int
((decl (innerVar) int))
()
((assign innerVar (literal 5))
(yield (identifier innerVar)))))
((assign outerVar (invoke Inner ()))
(yield (identifier outerVar))))
Your parser correctly recognized that Inner is declared inside Outer, just like your professor's ScopeCheck example.
Next fix: Standalone invoke statements
Your professor's grammar allows two different uses:
Plain text
invoke calculate(5, 10) => x;
invoke calculate(5, 10);
The first is an assignment using a routine call. The second calls a routine without
Ask ChatGPT
Instant

i scope example on page 2, and standalone `invoke` statements ... e Share I
page 3. CS 441 Program 2 Fall26.pdf CS 441 Program 2 Fall26.pdf

Toggle summary

What we need to fix before submission

1. Nested routines

Your current `parse-routine` reads variable declarations and then statements. It doesn't read additional routine declarations inside a routine.

But the professor's `ScopeCheck` example contains an `Inner` routine declared inside `Outer`.

This is a definite missing feature.

2. Standalone `invoke` statements

Your parser accepts:

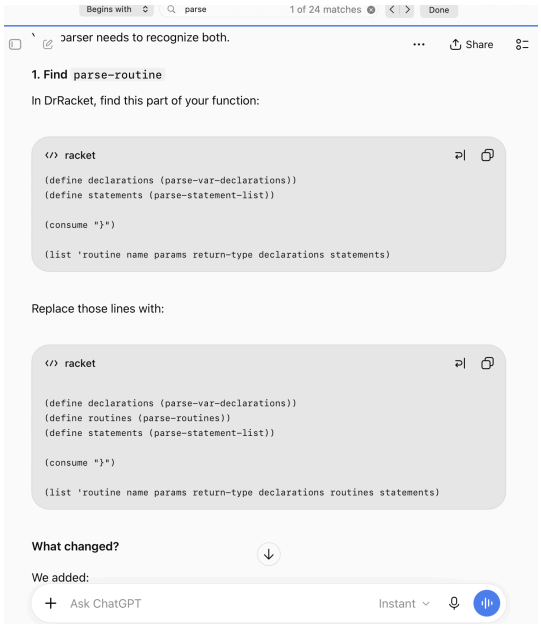
Plain text

```
invoke calculate(5, 10) => x;
```

But the grammar also allows:

Plain text

```
invoke calculate(5, 10);
```



Share

Let's break it down:

- `string-length` counts how many characters are in the text.
- `in-range` gives us the positions of those characters, starting at 0.
- `string-ref` reads a character at a specific position.
- `char=?` checks whether two characters match.
- `for/first` returns the position of the first matching character.

For example, add this temporary test:

racket

```
(string-index "Hello" #\l)
```

The result should be:

Plain text

```
2
```

Because Racket starts counting character positions at 0.



Share

CS 441 Program 2 Fall26(5).pdf
PDF

I attached the PDF for my program, before starting, can you help me understand what exactly I am suppose to build. Im kinda confused about how recursing descent parser works. Can you break it down.

Of course!! 😊 I went through your professor's PDF, and I can see why this assignment might be confusing at first. There are a lot of grammar rules, but the main idea is actually pretty straightforward.

1. What exactly are we building?

For Program 2, your professor wants you to build a **recursive descent parser using Racket**.

Think of it as a program that reads another program and checks whether it is written correctly.

For example, your professor's language allows:

Plain text

```
program Demo {
    decl x as int;
    5 => x;
}
```

Plain text

```
    5 => x;
}
```

Your parser needs to understand that:

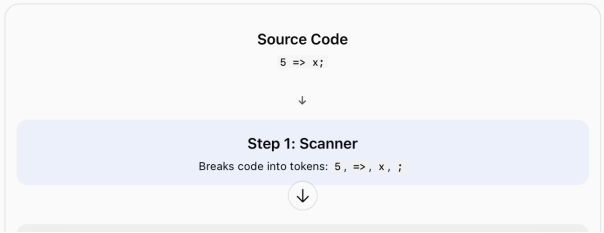
- `program Demo` starts a program named Demo.
- `decl x as int;` declares a variable named x.
- `5 => x;` assigns the value 5 to x.
- The braces `{ }` contain the program's code.

Your parser then turns that code into a structured representation called an **Abstract Syntax Tree (AST)**.

You are not being asked to build a full programming language from scratch or execute every statement. Your main job is to recognize the language's structure and produce Racket S-expressions.

2. How does a recursive descent parser work?

There are two main parts to this assignment.



What should I work on first? Scanner or Pars...

Sha

I recommend starting with the **scanner first**, then moving on to the parser.

The reason is that the parser depends on the scanner to break the source code into smaller pieces called **tokens**.

For example, if we have:

Plain text

```
5 => x;
```

The scanner breaks it into:

- `5` → Number
- `=>` → Operator
- `x` → Identifier
- `;` → Semicolon

Once we have those tokens, the parser can check whether they follow the grammar rules and create an AST.

Here's the order I recommend:

1. **Scanner** — Recognize numbers, identifiers, strings, operators, and comments.
2. **Token handling** — Keep track of the current token and move to the next one.
3. **Parser** — Start with simple grammar rules, then add expressions, assignments, loops, and routines.
4. **Testing** — Check valid and invalid programs.

Ask ChatGPT Instant

Yes, can you help me get started. im kinda lost with where to start

Share

Of course!! 😊 We'll build it **one small step at a time**, and I'll explain what each part does so you're not just copying code without understanding it.

Since your professor wants a recursive descent parser in Racket, we'll start with the **scanner**.

Step 1: Create your Racket file

First, open **DrRacket** and create a new file.

At the very top, type:

racket

```
#lang racket

; CS 441 – Program 2
; Recursive Descent Parser

; This program will read source code,
; break it into tokens, and check if
; it follows the grammar rules.
```

Save it as `Program2.rkt`.

Step 2: Create our token structure

Right below those comments, add:

Ask ChatGPT Instant

This represents a number token with the value `"5"` on line 1.

The `#:transparent` part lets us see the token's contents when we print it, which makes debugging easier.

Step 3: Test it

Underneath your code, add:

racket

```
(token 'number "5" 1)
```

Then click **Run** in DrRacket.

You should see:

Plain text

```
(token 'number "5" 1)
```

This is just our first test. We haven't built the scanner yet, but now we have a way to represent the tokens it will produce.

- `printf` displays information about each token.
- `token-type`, `token-value`, and `token-line` retrieve the three pieces of information stored in the token.

Let's test it!

Underneath the function, temporarily add:

```racket
(show-tokens
 (list (token 'integer "5" 1)
       (token 'symbol "=>" 1)
       (token 'identifier "x" 1)
       (token 'symbol ";" 1)))
```

Click **Run** in DrRacket.

You should see:

```
Type: integer | Value: 5 | Line: 1
Type: symbol | Value: => | Line: 1
Type: identifier | Value: x | Line: 1
Type: symbol | Value: ; | Line: 1
```

Share

Step 2: Create our token structure

Right below those comments, add:

```racket
(struct token (type value line) #:transparent)
```

What does this mean?

We're creating a structure called `token` that stores three things:

- **type:** What kind of token it is, such as a number or identifier.
- **value:** The actual text, such as `"5"` or `"x"`.
- **line:** The line number where the token appears, which helps us report syntax errors.

For example:

```racket
(token 'number "5" 1)
```

This represents a number token with the value `"5"` on line 1.

The `#:transparent` part lets us see the token's contents when we print it, which makes debugging easier.

Step 3: Test it

Underneath your code, add:

Ask ChatGPT Instant

Share

Added the token structure and it works. let start on the scanner. I dont really know how to separate everything into tokens.

Of course!! 😊 Let's start with the scanner, one small step at a time.

Remember, the **scanner's job is to read source code and break it into tokens**. Each token represents something meaningful, like a number, variable name, or operator.

For example, if our source code is:

Plain text

```
5 => x;
```

We want the scanner to recognize:

Type	Value	Line
integer	5	1
symbol	=>	1
identifier	x	1
symbol	;	1

Step 1: Create a function to display tokens

Before we build the scanner itself, let's make a small function that displays the tokens in a readable way. This will help us test our scanner later.

Ask ChatGPT Instant

symbol	;	1

Step 1: Create a function to display tokens

Before we build the scanner itself, let's make a small function that displays the tokens in a readable way. This will help us test our scanner later.

Under your token structure:

```racket
(struct token (type value line) #:transparent)
```

Add this function:

```racket
(define (show-tokens tokens)
  (for ([current-token tokens])
    (printf "Type: ~a | Value: ~a | Line: ~a\n"
            (token-type current-token)
            (token-value current-token)
            (token-line current-token))))
```

What is this code doing?

- `define` creates a function named `sh-tokens`.
- `for` goes through each token in a list.

Share

Works!, how do I actually start reading the source code and separating into token. how to do that in racket

Yess!! 😊 Now we're getting to the actual scanner!

In Racket, we can use **regular expressions (regex)** to recognize different parts of the source code, like numbers, variable names, and symbols.

But before writing the entire scanner, let's create one small helper function.

Step 2: Create `string-index`

This function will help us find the position of a character inside a string. We'll use it later when handling comments and strings.

Right below your `show-tokens` function, add:

```racket
(define (string-index text character)
  (for/first ([i (in-range (string-length text))]
              #:when (char=? (string-ref text i) character))
    i))
```

What does this function do?

Let's break it down:

- `string-length` counts how many characters are in the text.
- `in-range` gives us the positions of those characters, starting at 0.